Confidia

Architecture Technique

Document de cadrage fonctionnel - Clément Defer - Juin 2026

1. Définir l'architecture monolithe modulaire

1.1 Objectifs

L'architecture de **CONFIDIA** a été conçue pour répondre à un double impératif : permettre le développement rapide d'un premier produit exploitable tout en préservant une capacité d'évolution à long terme.

Compte tenu du contexte du projet, à savoir un MVP développé principalement par une équipe restreinte, une approche en monolithe modulaire a été retenue.

Cette décision vise à limiter la complexité opérationnelle inhérente aux architectures distribuées tout en garantissant une séparation claire des responsabilités fonctionnelles.

L'architecture doit notamment permettre :

- L'ajout progressif de nouvelles fonctionnalités
- L'intégration de services tiers sans remise en cause majeure de l'existant
- La prise en charge de plusieurs organisations indépendantes
- Le maintien d'une base de code cohérente et facilement maintenable
- Une éventuelle évolution vers une architecture plus distribuée si la croissance du produit le justifie

---

1.2 Architecture retenue

Le backend de CONFIDIA repose sur un monolithe modulaire développé avec **FastAPI**.

L'application est déployée comme une unité logique unique mais organisée en modules indépendants partageant les mêmes mécanismes techniques. Chaque module possède ses propres responsabilités, ses interfaces clairement définies et un périmètre fonctionnel maîtrisé.

Cette approche permet de bénéficier :

- De temps de développement réduits
- D'une simplicité de déploiement
- D'une meilleure productivité pour une équipe restreinte
- D'une dette opérationnelle limitée
- D'une migration facilitée vers des microservices si cela devient nécessaire

1.3 Modules internes

L'architecture applicative est structurée autour des modules suivants :

Module	Responsabilités principales
Auth	Authentification, gestion des sessions, contrôle d'accès
Documents	Gestion documentaire, classement, recherche, archivage
Messaging	Connexion aux messageries externes et synchronisation
Calendar	Gestion des calendriers et événements
AI	Analyse documentaire, traitements IA et orchestration des modèles
Payments	Gestion des paiements et transactions (évolution future)
Signatures	Signature électronique et workflows associés (évolution future)
Admin	Paramétrage, supervision et administration de la plateforme

Chaque module expose des interfaces internes clairement identifiées afin de limiter les dépendances implicites.

1.4 Services externes

CONFIDIA s'appuie sur plusieurs composants spécialisés afin d'éviter le développement de fonctionnalités déjà maîtrisées par des solutions éprouvées.

Service	Responsabilité
Keycloak	Authentification et fédération des identités
Paperless-ngx	Indexation et recherche documentaire
Nextcloud	Stockage et partage des fichiers
PostgreSQL	Persistance des données métier
Redis	Cache applicatif et limitation de débit
OpenAI	Traitements IA spécialisés
Claude	Analyse documentaire à contexte étendu
Mistral	Cas d'usage conversationnels et optimisation des coûts

Cette approche favorise l'indépendance technologique et réduit les risques liés au développement de composants non différenciants.

1.5 Composant transverse : Notifications

Les notifications constituent un mécanisme partagé utilisé par plusieurs modules de la plateforme.

Elles ne sont pas considérées comme un module métier autonome dans le cadre du MVP.

Leur objectif est de diffuser des informations nécessitant une attention particulière de l'utilisateur.

Canaux envisages

Canal	Statut
Notifications push Flutter	MVP
Emails transactionnels	MVP
SMS	Évolution future

Cas d'usage identifiés

- Rappels d'échéances administratives
- Événements calendaires importants
- Organisation incohérente des calendriers
- Alertes de sécurité
- Validation de signatures
- Paiements nécessitant une action utilisateur

1.6 Évolution future

L'architecture retenue ne constitue pas un **choix définitif** mais un **compromis adapté au contexte actuel du projet**.

Une évolution vers une architecture en microservices pourra être envisagée si plusieurs facteurs convergent :

- Augmentation significative du trafic
- Multiplication des équipes de développement
- Besoins de scalabilité indépendants selon les domaines fonctionnels
- Exigences opérationnelles plus avancées

Le découpage modulaire adopté dès le MVP vise précisément à préparer cette éventuelle transition sans imposer dès aujourd'hui une complexité disproportionnée.

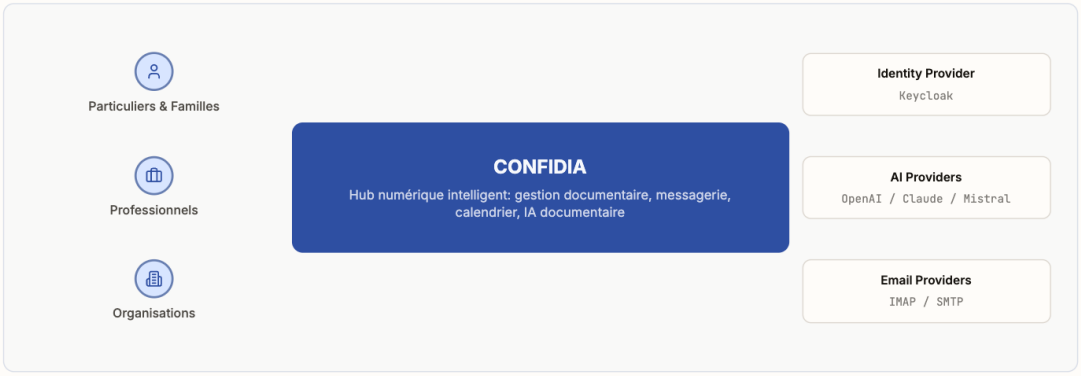
1.7 Philosophie et compromis du MVP

Certaines décisions architecturales ont été prises afin de privilégier la simplicité opérationnelle, la rapidité de mise sur le marché et la capacité de maintenance par une équipe réduite.

Le recours à un monolithe modulaire, à des composants open source spécialisés et à une infrastructure volontairement légère répond à cette logique.

Ces choix ne constituent pas des limitations structurelles du produit. Ils représentent des compromi, adaptés à la phase de lancement de **CONFIDIA** et susceptibles d'être réévalués à mesure que le produit, les usages et l'organisation évolueront.

C4 LEVEL 1 SYSTEM CONTEXT



C4 LEVEL 2 CONTAINER DIAGRAM

Monolithe Modulaire `FastAPI · Python`

auth	documents	messaging	calendar
ai	payments	signatures	admin

MVP Évolution future

⚡ **Notifications** (push, email, SMS futur): composant transverse partagé par tous les modules, pas un module métier autonome.

Frontend
Flutter
Mobile (iOS, Android) + Web

Auth	**Keycloak**	OAuth 2.0 / OIDC
Indexation	**Paperless-ngx**	OCR + FTS
Stockage	**Nextcloud**	WebDAV / S3
Database	**PostgreSQL**	Multi-tenant
Cache	**Redis**	Rate limit + cache
Monitoring	**Sentry + Uptime Kuma**	MVP stack

ARCHITECTURE DECISION RECORDS (HIGHLIGHTS)

Monolithe modulaire
Modules internes avec interfaces claires, pas de microservices.
Adapté à un dev solo + IA. Migration possible quand l'équipe et le trafic grandissent.

IA externe (cloud)
OpenAI, Claude, Mistral via Adapter Pattern avec routing et fallback.
Qualité maximale sans infra GPU. Modèles locaux envisagés si volume ou confidentialité l'exigent.

Infra simplifiée
Docker Compose + GitHub Actions + cloud EU (Scaleway).
Pas de K8s ni Terraform au MVP. Complexité opérationnelle minimale.

Open source first
Keycloak, Paperless-ngx, Nextcloud: composants matures et éprouvés.
Réduction des coûts, indépendance fournisseur, souveraineté des données.

Sécurité & RGPD
Chiffrement at-rest + in-transit, MFA, localisation UE, droit à l'effacement.
Conformité dès le jour 1: pas de dette de sécurité à rattraper.

Observabilité progressive
Sentry + Uptime Kuma au MVP, Prometheus + Grafana en évolution.
Investir le minimum viable en ops, scaler quand le produit le justifie.

PIPELINE DE DÉPLOIEMENT

DEV	→	CI/CD	→	HOSTING	→	MONITORING
Docker Compose		**GitHub Actions**		**Cloud EU**		**Sentry + Uptime**
Local, hot-reload		Build, test, deploy		Scaleway (ou equiv.)		Erreurs + disponibilité

2. Concevoir le schéma de base de données PostgreSQL

2.1 Objectifs

Le schéma de données de **CONFIDIA** doit permettre de supporter les besoins fonctionnels du MVP tout en préparant les évolutions futures du produit.

La modélisation retenue doit concilier plusieurs exigences parfois contradictoires : simplicité de mise en œuvre, bonnes performances, traçabilité des opérations et capacité à gérer plusieurs organisations indépendantes au sein d'une même plateforme.

L'objectif n'est pas de construire dès aujourd'hui un modèle exhaustif couvrant tous les cas d'usage futurs, mais de poser des fondations suffisamment robustes pour accompagner la montée en charge progressive du produit.

2.2 Principes de conception

Les choix de modélisation reposent sur les principes suivants :

Principe	Application dans CONFIDIA
Multi-tenant	Isolation logique des données par organisation
Traçabilité	Historisation des opérations sensibles et audit trail
Suppression logique	Utilisation du soft delete afin de préserver les données avant purge définitive
Performance	Indexation adaptée aux usages les plus fréquents
Évolutivité	Possibilité d'ajouter de nouveaux domaines fonctionnels sans remise en cause globale
Maintenabilité	Gestion des évolutions du schéma via Alembic

2.3 Domaines de données

Le modèle est organisé par domaines fonctionnels cohérents.

Domaine	Entités principales
Utilisateurs et organisations	users, organizations, memberships, roles
Gestion documentaire	documents, categories, tags, document_versions
Messagerie	mailboxes, emails, attachments
Calendrier	calendars, events, reminders
Intelligence artificielle	ai_analyses, ai_conversations
Paiements*	payments, invoices, transactions
Signature électronique*	signatures, workflows

* Évolutions prévues dans les phases ultérieures du projet.

Cette organisation favorise une lecture claire du modèle et s'aligne avec le découpage fonctionnel retenu au niveau applicatif.

2.4 Gestion du multi-tenant

CONFIDIA doit permettre à plusieurs espaces indépendants de coexister au sein d'une même infrastructure.

L'isolation des données repose sur une approche logique fondée sur la notion d'organisation.

Chaque donnée métier est rattachée à une organisation de référence, directement ou indirectement.

Les contrôles d'accès applicatifs garantissent qu'un utilisateur ne puisse consulter ou modifier que les informations appartenant aux espaces auxquels il est autorisé.

Cette approche présente plusieurs avantages :

- Simplicité de déploiement
- Mutualisation des ressources
- Évolutivité progressive
- Possibilité d'accueillir des particuliers, familles, associations ou entreprises au sein d'une architecture commune

---

2.5 Audit trail et conservation

Certaines opérations devront faire l'objet d'une traçabilité renforcée.

Les événements suivants seront notamment historisés :

- Création, modification ou suppression de documents
- Changements de rôles et de permissions
- Actions administratives sensibles
- Validations liées aux paiements ou signatures électroniques
- Traitements IA nécessitant une justification ultérieure

Par ailleurs, la suppression logique sera privilégiée grâce à des champs dédiés (date de suppression, auteur de l'action).

Cette approche répond à plusieurs objectifs :

- Restauration facilitée en cas d'erreur
- Respect des politiques de rétention
- Prise en compte des exigences réglementaires
- Préparation des futurs mécanismes d'audit

2.6 Performance et stratégie d'indexation

La stratégie d'optimisation vise avant tout les usages les plus fréquents du MVP.

Des index spécifiques seront mis en place sur les colonnes fortement sollicitées, notamment :

- Identifiants d'organisation
- Relations utilisateurs-organisations
- Dates d'événements
- Métadonnées documentaires
- États des traitements

Les optimisations plus avancées ne seront introduites qu'en fonction des besoins observés en production afin d'éviter une complexité prématurée.

2.7 Recherche documentaire

Paperless-ngx constitue le moteur principal d'indexation et de recherche documentaire de **CONFIDIA**.

Les capacités de recherche Full Text Search de PostgreSQL pourront néanmoins être utilisées de manière complémentaire lorsqu'elles apportent une réelle valeur ajoutée.

Le périmètre envisagé est le suivant :

Type de recherche	Mécanisme privilégié
Contenu documentaire OCR	Paperless-ngx
Recherche plein texte avancée sur les documents	Paperless-ngx
Recherche sur les noms de documents	PostgreSQL FTS
Recherche sur les tags et catégories	PostgreSQL FTS
Recherche sur certaines métadonnées	PostgreSQL FTS

Cette complémentarité permet de bénéficier des forces respectives de chaque composant sans dupliquer inutilement les responsabilités.

2.8 Gestion des évolutions

L'ensemble des évolutions du schéma sera piloté au moyen d'**Alembic**.

Chaque modification fera l'objet d'une migration versionnée permettant :

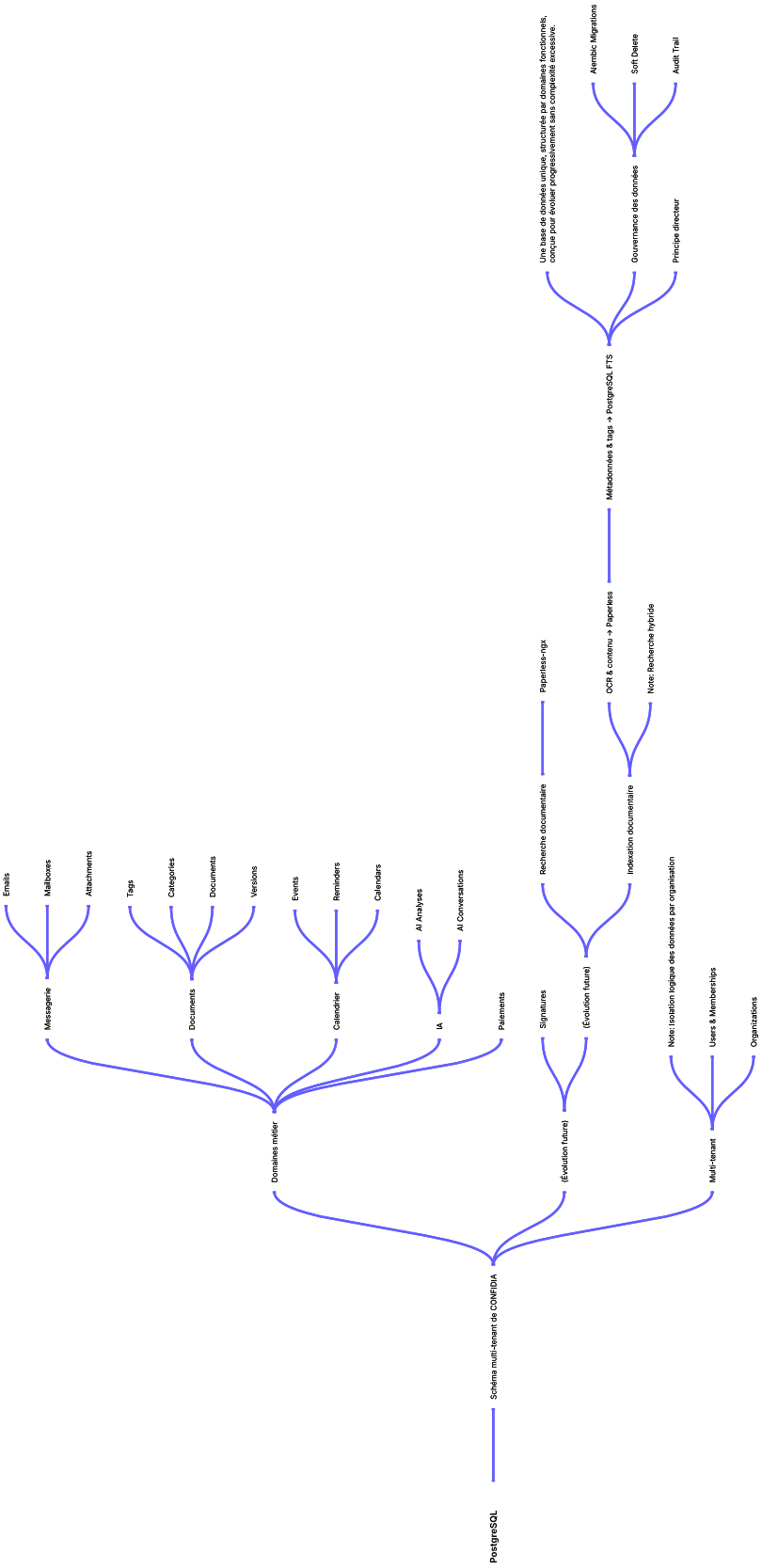
- De reproduire les environnements de manière fiable
- De sécuriser les déploiements
- De documenter l'historique des évolutions
- De faciliter les retours arrière lorsque cela est nécessaire

Cette approche garantit la cohérence du modèle de données tout au long du cycle de vie du produit.

2.9 Critères d'acceptation

La conception du schéma de données sera considérée comme validée lorsque :

- Le schéma SQL initial aura été défini
- Les contraintes d'intégrité auront été identifiées
- Le modèle multi-tenant aura été documenté
- Les mécanismes d'audit et de soft delete auront été intégrés
- La stratégie d'indexation aura été explicitée
- Les migrations **Alembic** initiales auront été préparées
- La répartition entre recherche **PostgreSQL** et indexation **Paperless-ngx** aura été clairement définie


PostgreSQL
Schéma multi-tenant de CONFIDIA
Domaines métier
Messagerie
Emails
Mailboxes
Attachments
Documents
Tags
Categories
Documents
Versions
Calendrier
Events
Reminders
Calendars
IA
AI Analyses
AI Conversations
Paiements
(Évolution future)
Signatures
Recherche documentaire
Paperless-ngx
(Évolution future)
Indexation documentaire
OCR & contenu → Paperless
Note: Recherche hybride
Métadonnées & tags → PostgreSQL FTS
Une base de données unique, structurée par domaines fonctionnels, conçue pour évoluer progressivement sans complexité excessive.
Multi-tenant
Note: isolation logique des données par organisation
Users & Memberships
Organizations
Gouvernance des données
Alembic Migrations
Soft Delete
Audit Trail
Principe directeur

3. Notifications transverses

3.1 Objectif

Le système de notifications a pour objectif d'informer les utilisateurs lorsqu'une action nécessite leur attention ou lorsqu'un événement important survient au sein de leur espace personnel ou organisationnel.

Ce mécanisme vise à améliorer la **réactivité des utilisateurs** et à **limiter les oublis** liés aux échéances administratives, aux événements calendaires ou aux actions nécessitant une validation.

Contrairement aux autres composants décrits dans cette architecture, les notifications ne constituent pas un module métier autonome. Elles représentent un **service transverse** utilisé par plusieurs domaines fonctionnels de la plateforme.

3.2 Canaux de diffusion

Plusieurs canaux de communication pourront être utilisés selon la nature de l'événement et les préférences de l'utilisateur.

Canal	Statut MVP	Description
Notifications push mobiles	Oui	Notifications envoyées vers les applications Flutter iOS et Android
Emails transactionnels	Oui	Notifications envoyées par courrier électronique
SMS	Évolution future	Canal complémentaire pour les notifications critiques

Chaque utilisateur pourra définir ses préférences de réception selon les types d'événements concernés.

3.3 Cas d'usage principaux

Le système de notifications pourra être sollicité par différents modules de la plateforme.

Origine	Exemple de notification
Calendrier	Rappel d'une échéance administrative ou d'un rendez-vous
IA documentaire	Détection d'une échéance dans un document
Signature électronique	Demande de signature ou validation effectuée
Paiements	Paiement à effectuer ou transaction validée
Sécurité	Connexion inhabituelle ou modification sensible du compte
Organisation	Attribution d'un nouveau rôle ou demande d'accès

---

3.4 Positionnement architectural

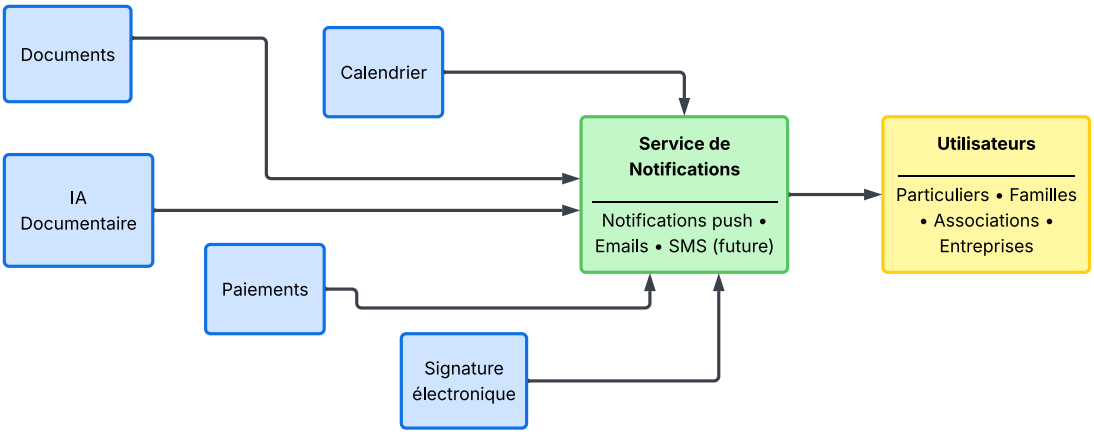
Les notifications sont conçues comme un service partagé accessible depuis **l'ensemble des modules applicatifs**.

Les différents domaines fonctionnels peuvent émettre des événements à destination du service de notifications, lequel se charge ensuite de sélectionner les canaux appropriés et de distribuer les messages aux utilisateurs concernés.

Cette approche permet :

- D'éviter la duplication de logique dans chaque module
- D'uniformiser les mécanismes de communication
- De faciliter l'ajout futur de nouveaux canaux de diffusion

Le système de notifications constitue ainsi une brique transverse de l'architecture sans porter de logique métier propre.


Documents
Calendrier
IA Documentaire
Service de Notifications
Notifications push • Emails • SMS (future)
Utilisateurs
Particuliers • Familles • Associations • Entreprises
Paiements
Signature électronique

4. Architecture API

4.1 Principes REST

L'ensemble des échanges entre les applications clientes et le backend CONFIDIA repose sur une architecture API REST.

Cette approche a été retenue pour sa simplicité de mise en œuvre, sa compatibilité avec l'écosystème Flutter et sa facilité d'intégration avec des services tiers. Elle permet également de générer automatiquement une documentation exploitable par les équipes de développement et les futurs partenaires.

Les ressources métier sont exposées sous forme d'endpoints clairement identifiés et manipulés via les verbes HTTP standards :

Verbe	Usage
GET	Consultation des données
POST	Création d'une ressource
PUT	Mise à jour complète
PATCH	Mise à jour partielle
DELETE	Suppression logique ou archivage

Les échanges utilisent exclusivement le format JSON.

4.2 Authentification JWT

La sécurisation des API repose sur **Keycloak**, utilisé comme fournisseur central d'identité.

Après authentification, l'utilisateur reçoit un **jeton JWT** contenant son identité, ses rôles et les informations nécessaires au contrôle des accès.

Le backend **FastAPI** valide systématiquement ces jetons avant l'exécution des opérations demandées.

Cette approche présente plusieurs avantages :

- Centralisation de l'authentification
- Compatibilité avec les fournisseurs externes (Google, Microsoft, Apple)
- Prise en charge native du MFA
- Gestion simplifiée des rôles et permissions

Les mécanismes détaillés d'authentification sont décrits dans le chapitre dédié à l'architecture Keycloak.

4.3 Convention de nommage

Les endpoints suivent des règles de nommage homogènes afin de garantir la lisibilité et la cohérence de l'ensemble de l'API.

Principes retenus :

Règle	Exemple
Noms au pluriel	/documents
URLs en minuscules	/organizations
Séparation par tirets si nécessaire	/document-categories
Utilisation des identifiants dans le chemin	/documents/{id}
Relations explicites	/organizations/{id}/members

Exemples d'endpoints :

```
GET     /documents
GET     /documents/{id}
POST    /documents
PATCH   /documents/{id}

GET     /organizations/{id}/members
POST    /organizations/{id}/members
```

L'objectif est de conserver une API intuitive, facilement compréhensible par tout développeur externe.

4.4 Pagination

Certaines ressources peuvent contenir un volume important de données, notamment les documents, les emails, les événements calendaires ou les conversations IA.

Afin de limiter la charge des requêtes et d'améliorer les performances, les listes sont paginées selon une approche cursor-based.

Cette méthode présente plusieurs avantages :

- Meilleure stabilité des résultats ;
- Performances constantes sur les grands volumes ;
- Compatibilité avec les usages mobiles.

Les paramètres de filtrage et de tri sont standardisés sur l'ensemble des ressources afin de garantir une expérience homogène.

4.5 Gestion des erreurs

Les erreurs renvoyées par l'API utilisent un format unique permettant aux applications clientes de les traiter de manière cohérente.

Structure générique :

```
{
  "code": "DOCUMENT_NOT_FOUND",
  "message": "Le document demandé est introuvable",
  "details": {}
}
```

Les codes HTTP standards sont utilisés :

Code	Signification
200	Succès
201	Ressource créée
400	Requête invalide
401	Authentification requise
403	Accès refusé
404	Ressource introuvable
409	Conflit métier
500	Erreur interne

L'objectif est de simplifier le traitement des erreurs côté client tout en facilitant le diagnostic des incidents.

4.6 Versionnement

Afin de préserver la compatibilité entre les différentes versions des applications clientes, l'API est versionnée dès sa première publication.

Le versionnement est intégré directement dans l'URL :

```
/api/v1/documents
/api/v1/calendars
/api/v1/organizations
```

Cette stratégie facilite la maintenance et permet l'introduction progressive d'évolutions majeures sans rupture pour les utilisateurs existants.

Les futures versions pourront coexister temporairement afin d'accompagner les migrations applicatives.

4.7 Documentation OpenAPI

L'ensemble des endpoints est documenté via la spécification **OpenAPI 3.1** générée automatiquement par **FastAPI**.

Cette documentation constitue la référence technique de l'API et permet :

- La consultation interactive des endpoints
- La génération de clients API
- Les tests manuels
- La synchronisation avec les outils de développement

La documentation est maintenue conjointement au code source afin de garantir sa cohérence avec les fonctionnalités réellement disponibles.

OpenAPI devient ainsi le contrat technique entre le backend **CONFIDIA**, les applications clientes et les futurs services partenaires.

5. Architecture d'authentification

5.1 Keycloak

L'authentification de **CONFIDIA** repose sur **Keycloak**, solution open source reconnue pour la gestion centralisée des identités et des accès.

L'utilisation d'un fournisseur d'identité dédié permet de séparer les problématiques de sécurité et d'authentification du code métier de l'application. Cette approche simplifie la maintenance, améliore la sécurité globale du système et facilite l'intégration de nouveaux mécanismes d'authentification à l'avenir.

Un realm dédié nommé **CONFIDIA** est mis en place afin d'isoler l'ensemble des utilisateurs, rôles, permissions et paramètres de sécurité de la plateforme.

Les principaux clients déclarés dans Keycloak sont :

Client	Description
Flutter Mobile	Application iOS et Android
Flutter Desktop	Application desktop
API Backend	Services FastAPI
Administration	Outils internes de gestion

Keycloak devient ainsi le point d'entrée unique de toutes les opérations d'authentification de la plateforme.

5.2 Identity Providers

Afin de simplifier l'inscription et la connexion des utilisateurs, plusieurs fournisseurs d'identité externes sont intégrés dès les premières phases du projet.

Les fournisseurs initialement prévus sont :

Fournisseur	Usage
Google	Comptes personnels et professionnels
Microsoft	Outlook, Microsoft 365
Apple	Utilisateurs iOS et macOS

Cette approche permet aux utilisateurs de se connecter à l'aide de comptes déjà existants sans devoir gérer un mot de passe supplémentaire.

Elle facilite également l'intégration des services connectés utilisés dans d'autres modules de la plateforme, notamment la messagerie et les calendriers.

L'ajout de nouveaux fournisseurs d'identité reste possible grâce aux mécanismes standards pris en charge par Keycloak.

5.3 Gestion des rôles

Le contrôle des accès repose sur un modèle de rôles hiérarchisés.

Chaque utilisateur peut appartenir à une ou plusieurs organisations et disposer de permissions différentes selon son contexte d'utilisation.

Les rôles principaux identifiés pour le MVP sont les suivants :

Rôle	Description
User	Utilisateur standard
Org Admin	Administrateur d'une organisation
Super Admin	Administration globale de la plateforme

Cette organisation répond à la nature multi-tenant de CONFIDIA, dans laquelle plusieurs structures indépendantes peuvent coexister sur une même infrastructure.

À terme, certains espaces pourront définir des rôles spécifiques adaptés à leur fonctionnement :

- Foyer familial
- Association
- Entreprise
- Organisation sportive
- Structure professionnelle réglementée

Les contrôles d'accès sont systématiquement vérifiés côté backend avant toute opération sensible.

5.4 MFA

La plateforme manipule des données administratives, financières et documentaires potentiellement sensibles.

Pour cette raison, une authentification multi-facteurs (MFA) est prévue dans l'architecture dès les premières phases du projet.

Le MFA pourra être activé de manière facultative pour les utilisateurs particuliers et rendu obligatoire pour certains contextes plus sensibles :

- Administrateurs d'organisation
- Accès à des documents sensibles
- Opérations financières
- Signature électronique
- Actions d'administration

Les mécanismes supportés dépendront des fonctionnalités disponibles dans Keycloak au moment du déploiement :

- Application d'authentification
- Code temporaire
- Validation sur appareil de confiance

Cette approche permet d'augmenter significativement le niveau de sécurité sans complexifier inutilement l'expérience utilisateur standard.

5.5 Gestion des tokens

Les sessions utilisateurs reposent sur le mécanisme standard OAuth2 / OpenID Connect fourni par Keycloak.

Deux types de jetons sont utilisés :

Jeton	Fonction
Access Token	Autorisation des appels API
Refresh Token	Renouvellement de session

Les Access Tokens possèdent une durée de vie courte afin de limiter les risques en cas de compromission.

Les Refresh Tokens permettent quant à eux de maintenir une session utilisateur active sans nécessiter une réauthentification fréquente.

Le backend FastAPI vérifie la validité des jetons lors de chaque requête protégée.

Les informations contenues dans les tokens permettent notamment de :

- Identifier l'utilisateur
- Connaître son organisation de rattachement
- Déterminer ses rôles
- Appliquer les règles d'autorisation correspondantes

6. Architecture documentaire

6.1 Nextcloud

Nextcloud constitue la couche principale de stockage documentaire de **CONFIDIA**.

Ses responsabilités sont les suivantes :

- Stockage des fichiers utilisateurs
- Gestion de l'arborescence documentaire
- Partage de documents
- Synchronisation entre appareils
- Gestion des quotas de stockage

Nextcloud est considéré comme la source de vérité des fichiers déposés dans la plateforme.

6.2 Paperless-ngx

Paperless-ngx intervient comme moteur documentaire spécialisé.

Ses responsabilités sont les suivantes :

- Indexation des documents
- OCR et extraction de texte
- Catégorisation documentaire
- Recherche full-text
- Enrichissement des métadonnées

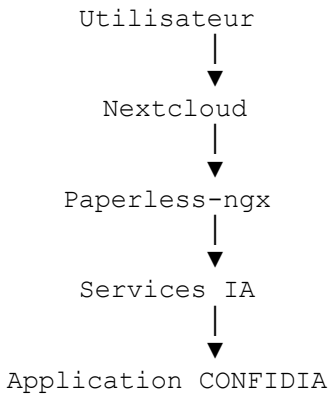
Paperless-ngx ne remplace pas le stockage documentaire mais apporte les capacités avancées de gestion et de recherche.

6.3 Flux documentaire

Le cycle documentaire retenu pour le MVP est le suivant :

1. Réception ou dépôt d'un document.
2. Stockage du fichier dans Nextcloud.
3. Transmission à Paperless-ngx pour indexation.
4. Analyse documentaire éventuelle par les services IA.
5. Mise à disposition du document dans l'application.

Cette organisation permet de séparer clairement les responsabilités entre stockage, indexation et analyse.



6.4 Sauvegardes

Les sauvegardes constituent un élément critique de l'architecture.

Les composants suivants doivent être sauvegardés :

- Fichiers stockés dans Nextcloud
- Base de données PostgreSQL
- Configuration Keycloak
- Configuration applicative

Les sauvegardes sont réalisées de manière automatisée et conservées dans un environnement distinct de l'infrastructure principale.

L'ensemble des données sauvegardées demeure hébergé au sein de **l'Union Européenne**.

6.5 Chiffrement

Les documents manipulés par **CONFIDIA** pouvant contenir des informations sensibles, plusieurs niveaux de protection sont prévus.

Les échanges réseau sont protégés par **TLS**.

Les données stockées bénéficient d'un chiffrement au repos mis en œuvre au niveau de l'infrastructure d'hébergement.

Les secrets techniques, clés API et identifiants d'accès sont stockés séparément des données métier.

6.6 Politique de rétention

La conservation des données doit respecter les exigences réglementaires et les besoins des utilisateurs.

Le système prévoit :

- La suppression logique des données (soft delete)
- Des durées de conservation configurables
- La restauration possible pendant une période définie
- La suppression définitive à l'issue de la période de rétention

Cette approche facilite la conformité RGPD tout en limitant les risques de suppression accidentelle.

7. Architecture IA

7.1 Objectifs

L'intelligence artificielle constitue l'un des éléments différenciants majeurs de **CONFIDIA**.

Son rôle est d'assister les utilisateurs dans la compréhension, l'organisation et l'exploitation de leurs informations sans remplacer leur capacité de décision.

L'architecture IA doit permettre :

- L'analyse documentaire
- L'extraction d'informations structurées
- La détection d'échéances et d'actions à réaliser
- L'assistance conversationnelle
- L'optimisation organisationnelle des calendriers individuels et collectifs

L'ensemble des recommandations produites par l'IA reste soumis à une validation humaine lorsqu'une action impacte plusieurs utilisateurs ou modifie des données métier.

7.2 Couche d'abstraction LLM

CONFIDIA repose sur plusieurs fournisseurs d'intelligence artificielle.

Afin d'éviter toute dépendance à un modèle unique, une couche d'abstraction est placée entre l'application et les différents LLM.

Cette couche a pour responsabilités :

- Uniformiser les appels aux modèles
- Standardiser les formats de réponse
- Simplifier l'ajout ou le remplacement d'un fournisseur
- Centraliser les règles de sécurité et de journalisation

L'application dialogue ainsi avec une interface unique indépendamment du fournisseur réellement utilisé.

7.3 Router IA

Un mécanisme de routage sélectionne automatiquement le modèle le plus adapté à chaque tâche.

Les critères de sélection peuvent notamment inclure :

- La qualité attendue
- Le coût d'exécution
- La taille du contexte
- La latence
- La disponibilité du fournisseur

À titre indicatif :

Cas d'usage	Modèle privilégié
Analyse documentaire	Claude
Extraction structurée	GPT
Conversation utilisateur	Claude / Mistral
Classification simple	Modèle léger

Cette stratégie permet d'optimiser simultanément la qualité des résultats et les coûts d'exploitation.

7.4 Fallback

Afin de garantir la continuité de service, un mécanisme de secours est prévu.

En cas d'indisponibilité, d'erreur ou de dépassement de quota d'un fournisseur, le système peut automatiquement basculer vers un autre modèle compatible.

Cette logique de fallback permet :

- D'améliorer la résilience globale
- De limiter les interruptions de service
- D'éviter une dépendance critique à un acteur unique

Le comportement de bascule reste transparent pour l'utilisateur final.

7.5 Gestion des coûts

L'utilisation des modèles d'intelligence artificielle représente un coût variable directement lié à l'activité de la plateforme.

Une surveillance spécifique est donc mise en place afin de suivre :

- La consommation par utilisateur
- La consommation par organisation
- Le coût par fonctionnalité
- Le coût par fournisseur

Ces indicateurs permettront d'ajuster les modèles utilisés et d'anticiper les besoins futurs de facturation ou de limitation d'usage.

7.6 Cache

Certaines opérations peuvent produire des résultats identiques sur une période donnée.

Afin de limiter les appels inutiles aux modèles, un mécanisme de cache est mis en place à l'aide de Redis.

Le cache permet notamment :

- De réduire les coûts
- D'améliorer les temps de réponse
- De limiter la charge sur les fournisseurs IA

La durée de conservation des résultats est configurable selon la nature des traitements effectués.

7.7 Évolutions futures

L'architecture retenue est conçue pour évoluer progressivement sans remise en cause majeure.

Les évolutions envisagées sont notamment :

- L'intégration de modèles open source locaux
- Le déploiement de modèles via Ollama ou vLLM
- L'amélioration du moteur d'IA organisationnelle
- L'ajout de nouvelles capacités d'analyse métier
- L'optimisation continue du rapport coût / performance

8. Infrastructure et DevOps

8.1 Hébergement

L'infrastructure du MVP est conçue pour privilégier la simplicité d'exploitation et la maîtrise des coûts.

L'hébergement est prévu auprès d'un fournisseur cloud européen afin de répondre aux exigences de localisation des données et de conformité réglementaire.

L'infrastructure héberge les principaux composants de la plateforme :

- Backend FastAPI
- PostgreSQL
- Redis
- Keycloak
- Nextcloud
- Paperless-ngx
- services de supervision

Cette architecture permet de démarrer rapidement tout en conservant des possibilités d'évolution ultérieures.

8.2 Docker

L'ensemble des composants applicatifs est conteneurisé à l'aide de Docker.

Cette approche garantit :

- La reproductibilité des environnements
- La simplification des déploiements
- L'isolation des services
- La portabilité entre hébergeurs

Pour le MVP, l'orchestration repose sur Docker Compose, suffisamment adapté au volume attendu et à une équipe de développement réduite.

8.3 CI/CD

Les déploiements sont automatisés à l'aide de GitHub Actions.

Le pipeline CI/CD a pour objectif de :

- Exécuter les contrôles qualité
- Lancer les tests automatisés
- Construire les images Docker
- Déployer les nouvelles versions

Cette automatisation réduit les risques d'erreur humaine et facilite la mise en production régulière des évolutions.

8.4 Gestion des secrets

Les informations sensibles ne sont jamais stockées dans le code source.

Les principaux secrets concernés sont :

- Clés API des fournisseurs IA
- Identifiants de bases de données
- Certificats
- Jetons d'accès externes

Ces éléments sont gérés via des variables d'environnement sécurisées et des mécanismes dédiés à la protection des secrets.

8.5 Environnements

L'architecture distingue au minimum deux environnements :

Environnement	Objectif
Développement	Conception et validation des fonctionnalités
Production	Utilisation réelle de la plateforme

Un environnement de préproduction pourra être ajouté ultérieurement si les besoins de validation deviennent plus importants.

Cette organisation permet de séparer les phases de développement des usages réels de la plateforme.

8.6 Scalabilité future

L'architecture retenue pour le MVP privilégie volontairement la simplicité opérationnelle.

Toutefois, plusieurs évolutions restent envisageables sans remise en cause majeure :

- Augmentation des ressources serveurs
- Séparation progressive de certains composants
- Ajout de nouveaux nœuds applicatifs
- Mise en place d'une orchestration Kubernetes
- Automatisation avancée de l'infrastructure

Cette approche permet d'accompagner la croissance du produit tout en évitant une complexité prématurée lors des premières phases du projet.

9. Sécurité et conformité

9.1 Chiffrement

La protection des données constitue une exigence fondamentale de CONFIDIA en raison de la nature potentiellement sensible des informations manipulées.

Deux niveaux de chiffrement sont mis en œuvre :

Niveau	Objectif
Chiffrement en transit	Protection des échanges réseau via TLS
Chiffrement au repos	Protection des données stockées et des sauvegardes

Les documents, informations personnelles et données métier bénéficient ainsi d'un niveau de protection cohérent avec les exigences du projet.

9.2 Gestion des accès

Le contrôle des accès repose sur les mécanismes d'authentification et d'autorisation fournis par Keycloak.

Les principes retenus sont :

- Authentification centralisée
- Gestion des rôles
- Authentification multi-facteurs
- Principe du moindre privilège

Chaque utilisateur accède uniquement aux données et fonctionnalités correspondant à son rôle et à son organisation de rattachement.

Cette approche limite les risques d'accès non autorisés tout en facilitant l'administration de la plateforme.

9.3 Journalisation

Les événements importants de la plateforme sont enregistrés afin de faciliter le suivi des activités et l'analyse des incidents.

Les informations journalisées peuvent notamment concerner :

- Les connexions et déconnexions
- Les opérations d'administration
- Les accès aux documents sensibles
- Les actions réalisées par les services IA
- Les erreurs applicatives

Les journaux sont conservés selon une politique définie et accessibles uniquement aux personnes autorisées.

9.4 RGPD

L'architecture est conçue pour faciliter le respect des principales obligations du Règlement Général sur la Protection des Données (*RGPD*).

Les mécanismes prévus incluent notamment :

- L'information et le consentement des utilisateurs
- L'export des données personnelles
- La suppression des données sur demande
- La gestion des durées de conservation
- L'hébergement des données au sein de l'Union Européenne

Certaines obligations complémentaires pourront faire l'objet d'une étude juridique spécifique lors des phases ultérieures du projet.

9.5 Audit

Afin de garantir la traçabilité des actions sensibles, un mécanisme d'audit est intégré à l'architecture.

Les opérations suivantes doivent pouvoir être historisées :

- Modifications des droits d'accès
- Actions administratives
- Opérations documentaires sensibles
- Traitements IA ayant produit une action utilisateur
- Opérations de paiement ou de signature électronique

L'objectif est de permettre la reconstitution des événements en cas d'incident, d'erreur ou de contrôle réglementaire.

Cette capacité de traçabilité constitue un élément essentiel de confiance pour les utilisateurs particuliers comme pour les organisations professionnelles.

10. Observabilité et exploitation

10.1 KPI techniques

Plusieurs indicateurs techniques sont suivis afin d'évaluer la qualité de fonctionnement de la plateforme.

Indicateur	Objectif
Disponibilité	Mesurer la continuité du service
Temps de réponse	Évaluer les performances
Taux d'erreur	Identifier les anomalies
Volume documentaire	Suivre la croissance des usages
Consommation IA	Contrôler les coûts d'exploitation

Ces indicateurs constituent la base du pilotage technique de la plateforme.

10.2 Gestion des incidents

Une procédure simple de gestion des incidents est prévue dès le MVP.

Les étapes principales sont les suivantes :

1. Détection de l'incident
2. Qualification du problème
3. Correction ou mise en œuvre d'une solution de contournement
4. Vérification du retour à la normale
5. Analyse des causes si nécessaire

Cette démarche permet de limiter les interruptions de service et d'améliorer progressivement la fiabilité de la plateforme.

Conclusion

L'architecture technique retenue pour **CONFIDIA** vise avant tout l'équilibre entre ambition fonctionnelle, maîtrise de la complexité et capacité de réalisation.

Le choix d'un **monolithe modulaire** reposant sur des composants **open source éprouvés** permet de construire rapidement un MVP robuste tout en conservant des perspectives d'évolution à long terme. Les principales briques techniques ont été sélectionnées pour leur maturité, leur indépendance technologique et leur adéquation avec les exigences de sécurité, de conformité et de maintenabilité du projet.

Cette architecture fournit ainsi un cadre cohérent pour les premières phases de développement tout en préparant l'intégration progressive des fonctionnalités avancées qui constituent la vision de **CONFIDIA** : intelligence artificielle documentaire, organisationnelle et conversationnelle, automatisation administrative, paiements, signature électronique et gestion collaborative des organisations.